

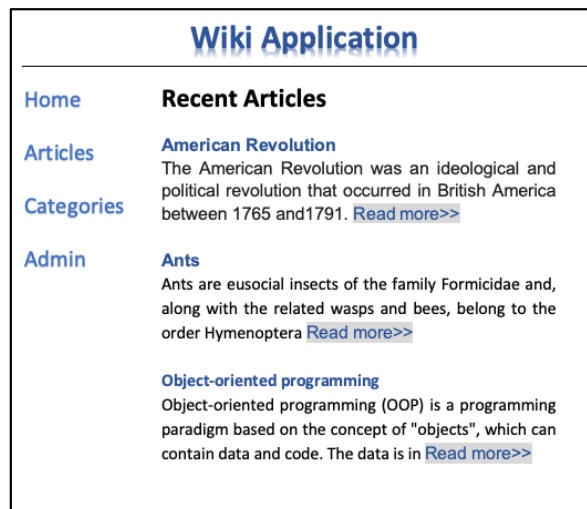
Assignment Part B – Requirements

Wiki Content Management and Admin Console

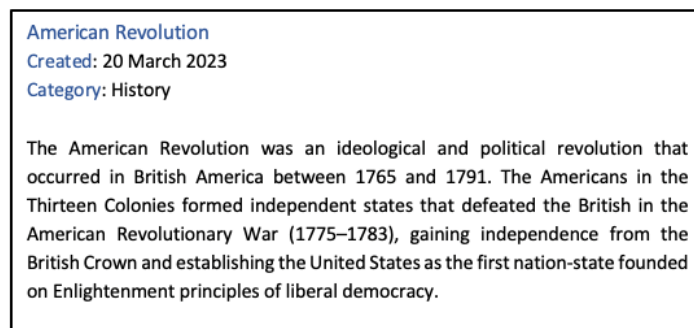
Functional requirements

Requirements for the system:

- Every page should contain a navigation pane. At minimum, the navigation should contain links to Home, Articles, Categories, and Admin. A link for logging out should be added when a client has logged in.
- A home page that shows a preview of recent articles (for example, the first 120–150 characters), with a 'Read more' link. An example of how it can be structured:



- A single article may be displayed like this, and the Category name should be a link to articles in that category:



- An admin-only area that is available only after login. Admin pages must check login status before allowing access. If a user is not logged in, they should be redirected to the login page:

Username

Password

[Register](#) [Log in](#)

- At least one complete CRUD workflow – either for Articles or for Categories

All Articles

American Revolution [Edit | Delete](#)
 The American Revolution was an ideological and political revolution that occurred in British America between 1765 and 1791. [Read more>>](#)

Ants [Edit | Delete](#)
 Ants are eusocial insects of the family Formicidae and, along with the related wasps and bees, belong to the order Hymenoptera [Read more>>](#)

Object-oriented programming [Edit | Delete](#)
 Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data and code. The data is in [Read more>>](#)

[Create New Article](#)
[Create New Category](#)
[Logout](#)

- A client not logged in can view articles and categories but not edit.
- A client logged in can perform CRUD operations.

CRUD requirements

Your system must demonstrate complete CRUD workflow for **either** Articles or Categories. That is, you need **not** implement CRUD operations for both Articles and Categories.

Only an admin (that is, a user logged in) should be able to Create, Update, or Delete database content.

Ask for confirmation to save changes and delete. For example:

Delete Page?

Are you sure you want to delete this article?
 This action cannot be undone.

[Cancel](#) [Delete](#)

Technical requirements

Your application must:

- Demonstrate basic **password hashing**. The admin password must not be stored or compared as plain text. When a user logs in, the submitted password should be hashed or checked against a stored hash before access is granted. You are not required to build enterprise-level security, but you must be able to explain why password hashing is used and how your login check works.
- Extend the Spring MVC structure introduced in Part 1, by including **Service** and **Repository**.

- Follow **standard Spring Boot and MVC conventions** for file structure, class organisation, and code placement, including appropriate separation of Controllers, Services, Repositories, Models, templates, and static resources.
- Use **names**, such as method, class, and variable names, that make sense for a wiki-style application.
- Add **comments** to your code to show that you understand the code that you are submitting and to help others understand what you have written.
- Use **sessions** to maintain login status across multiple requests so that admin remains logged in while navigating your application.
- Use Thymeleaf **fragments** where sensible (especially for header/navigation and footer)
- Add **a test record** to the admin table with a username of 1 and a password of 1. The password must be stored as a hash (using the same hashing approach as the other admin records). This account is provided only to make testing and marking easier and would be removed if your application was going into production.
- Include **your own name**, identified as the 'web master' in the footer of all pages.
- Minimise server calls by making use of at least one **HTML** and one **JavaScript validation** technique.
- Include at least two meaningful **JUnit tests** for important application logic, such as login validation, password checking, article creation, category handling, or validation of empty/invalid input. These tests do not need to cover the whole application, but they should demonstrate that you understand how to test part of your system.

Model, repository, and data storage

Your application should include the following model/entity classes regardless of storage approach:

- Admin
- Category
- Article

Each Article can belong to only one Category, and a Category can contain many Articles.

Your Repository layer may use one of the following approaches:

- An in-memory data structure, such as an ArrayList or HashMap
- A database. This can be implemented with a write-to-disk database like MySQL Workbench, or the write-to-memory H2 database.

Static resources and presentation

Your application must include:

- at least one external CSS file
- at least one logo image
- consistent navigation across pages
- a clear and readable visual layout

Static resources must be stored in appropriate Spring Boot static folders such as:

- static/css
- static/images
- static/js

While the application should appear organised, readable, and professionally presented, students are not assessed on advanced graphic design or advanced CSS techniques.

AI-assisted CSS generation and the use of CSS templates or Bootstrap themes are permitted.

Security

Complex security is not required. However, your application should demonstrate some of the basic security practices implemented in workshops this semester. These include protecting admin pages with sessions, avoiding plain-text password storage, reducing SQL injection risk through appropriate repository/database handling, and displaying user-provided content safely to reduce XSS risk. HTML and JavaScript validation should also be used to reduce invalid input before submission.